
Mobile App Pentesting

AndroGoat Security Assessment Walkthrough

A beginner-friendly personal lab writeup combining network interception, static analysis, exported component testing, emulator rooting and Frida runtime bypass.

Prepared by **Mehdi Abouelmouahib**

Project	Personal Android Mobile Security Lab
Author	Mehdi Abouelmouahib
Target	AndroGoat - intentionally vulnerable Android application
Focus	Burp Suite, JADX, ADB, rootAVD, Frida Server
Scope	Controlled lab and educational testing only

What this writeup covers

- Android lab setup and APK installation with ADB
- Network interception workflow with Burp Suite
- Static analysis of the APK, Manifest and exported components
- Rooted emulator setup, root-detection concepts and Frida runtime bypass

Responsible use

This work is documented as a controlled personal lab on an intentionally vulnerable application only.

Roadmap

This document rewrites the original lab reports into a single portfolio-style walkthrough. The school/report framing has been removed and the narrative is written as a personal security project.

1. Project introduction and scope
2. Lab setup and installation
3. Network interception with Burp Suite
4. Static analysis with JADX-GUI
5. Exported Android components
6. Hardcoded secrets and local data exposure
7. Rooting the emulator with rootAVD
8. Root detection theory
9. Bypassing root detection with Frida
10. Findings, mitigations and lessons learned

Responsible use

All steps are intended for an intentionally vulnerable lab application and an emulator controlled by the tester. The same workflow must not be used on third-party apps or real services without explicit authorization.

1. Project introduction and scope

Mobile applications can expose risk through many layers: the APK itself, the AndroidManifest, exported components, local storage, network configuration and runtime checks. In this project, I used AndroGoat as a safe training target to practice a complete Android mobile pentesting workflow.

The goal was not to run one tool and stop. The goal was to move like a real mobile security tester: prepare the lab, observe traffic, inspect the APK, test external entry points, look for secrets, root the emulator, then use dynamic instrumentation to understand how runtime security checks can be manipulated.

Main learning objectives

- Build a working Android testing lab with ADB, an emulator, Burp Suite and JADX-GUI.
- Intercept HTTP traffic from the Android emulator and understand why proxy/certificate setup matters.
- Read the AndroidManifest.xml to identify risky flags, exported components and deeplinks.
- Search decompiled code for hardcoded secrets, credentials and weak local storage patterns.
- Root an Android Studio emulator using rootAVD and verify the result from both the app and ADB.
- Bypass a client-side root detection mechanism with Frida Server and a custom JavaScript hook.

2. Lab setup and installation

The lab was built around an Android Studio emulator and a vulnerable APK. The target package used throughout the project is `owasp.sat.agoat`.

Tool	Role in the lab
Android Studio Emulator	Runs the target Android device in a controlled environment.
ADB	Installs APKs, opens shells, configures proxy settings and starts Android components.
Burp Suite	Intercepts and inspects HTTP/HTTPS traffic.
JADX-GUI	Decompiles the APK and helps review code, manifest and resources.
rootAVD	Enables root on the Android Studio emulator.
Frida + Frida Server	Hooks methods dynamically inside the running Android process.

2.1 Verifying the emulator and installing the APK

Before testing the application, I confirmed that ADB could see the emulator and then installed the debug APK.

```
adb devices
adb install -r app-debug.apk
adb shell pm list packages | findstr /i goat
```

Expected result

The installation returns Success, and the package `owasp.sat.agoat` appears in the package list.

```
PS C:\Users\VIET\Downloads\AndroGoat-master\AndroGoat-master> adb install -r "C:\Users\VIET\Downloads\AndroGoat-master\AndroGoat-master\app\build\outputs\apk\debug\app-debug.apk"
Performing Streamed Install
Success
PS C:\Users\VIET\Downloads\AndroGoat-master\AndroGoat-master> adb shell pm list packages | findstr /i goat
package:owasp.sat.agoat
PS C:\Users\VIET\Downloads\AndroGoat-master\AndroGoat-master> adb devices
List of devices attached
emulator-5554    device
PS C:\Users\VIET\Downloads\AndroGoat-master\AndroGoat-master>
```

Figure 3. ADB detects the emulator and confirms the AndroGoat package installation.

2.2 Launching AndroGoat

Once installed, AndroGoat exposes multiple vulnerable modules. The main menu is useful because it gives a map of the attack surface: network interception, unprotected components, insecure data storage, hardcoded secrets, root detection and more.

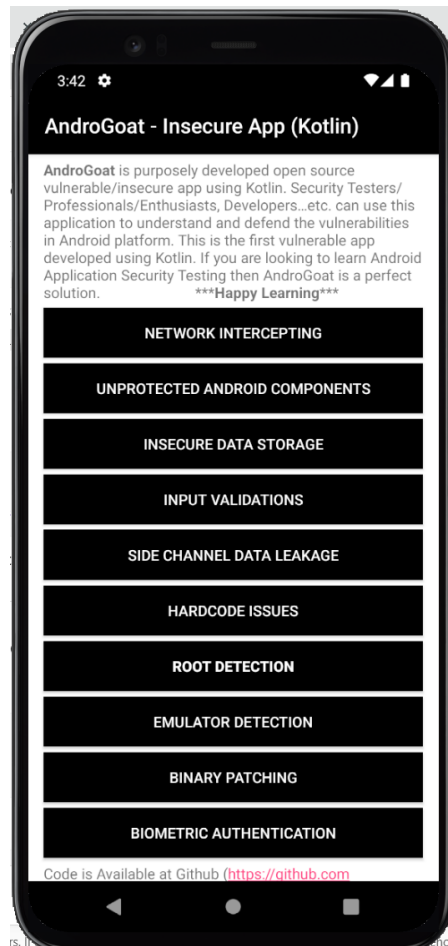


Figure 4. AndroGoat main screen showing the vulnerable modules used in this assessment.

3. Network interception with Burp Suite

Network interception is usually one of the first dynamic tests in a mobile assessment. It helps answer a simple question: what does the application send or receive when I interact with it?

3.1 Configuring the proxy

Burp Suite was running on a Kali machine, so the proxy listener had to accept traffic from outside localhost. I configured Burp to listen on all interfaces on port 8080, then configured the emulator proxy through ADB.

```
adb shell settings put global http_proxy 192.168.56.101:8080
adb shell settings get global http_proxy
```

I also verified that the Burp listener was reachable from the host side:

```
Test-NetConnection 192.168.56.101 -Port 8080
```

Why this matters

If the emulator cannot reach Burp, no interception will happen. Debugging the network path early saves time before touching the application itself.

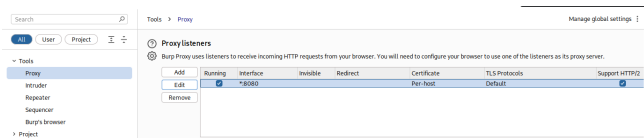


Figure 5. Burp listener configured on port 8080.

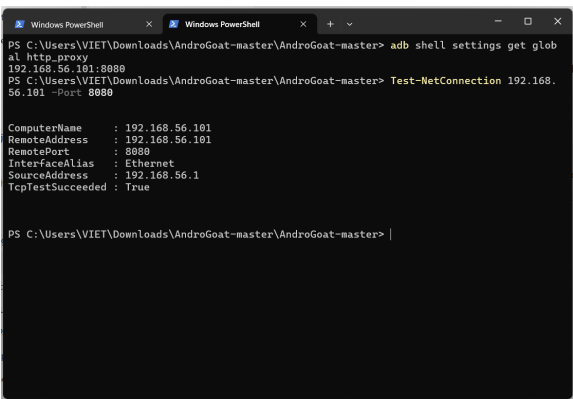


Figure 6. Connectivity test confirming that the proxy is reachable.

3.2 Installing the Burp CA certificate

For HTTPS interception, the emulator must trust the Burp CA certificate. In this lab, I installed the certificate inside Android and confirmed that the CA was accepted.

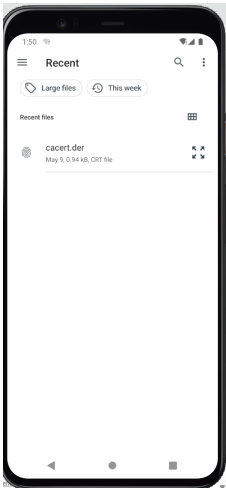


Figure 7. Burp certificate file present inside the emulator.

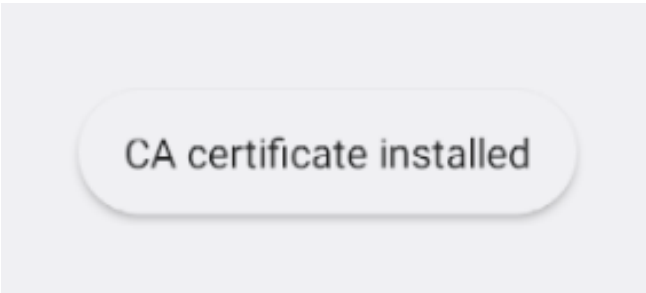


Figure 8. Android confirmation: CA certificate installed.

3.3 Testing HTTP interception

Inside AndroGoat, I opened the network interception module and triggered the HTTP request. The request then appeared in Burp Suite's HTTP history, proving that the proxy configuration worked.



Figure 9. HTTP test button inside AndroGoat.

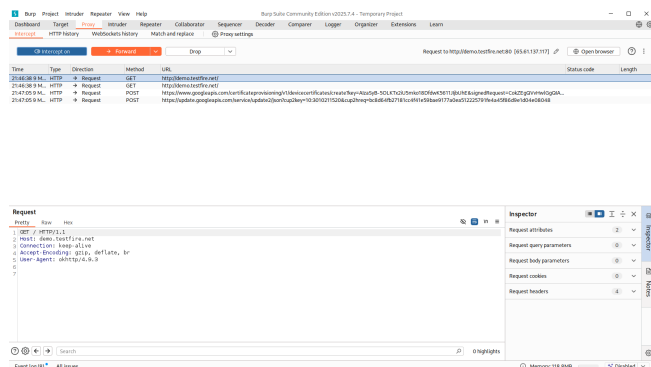


Figure 10. The HTTP request is visible inside Burp Suite.

Security lesson

In production, sensitive data must not be sent over cleartext HTTP. Even when HTTPS is used, release builds should avoid trusting user-installed CAs unless there is a specific enterprise use case.

4. Static analysis with JADX-GUI

Static analysis means studying the APK without relying only on what the UI shows. JADX-GUI allows the tester to inspect decompiled Java/Kotlin code, resources and the AndroidManifest.xml.

4.1 Opening the APK

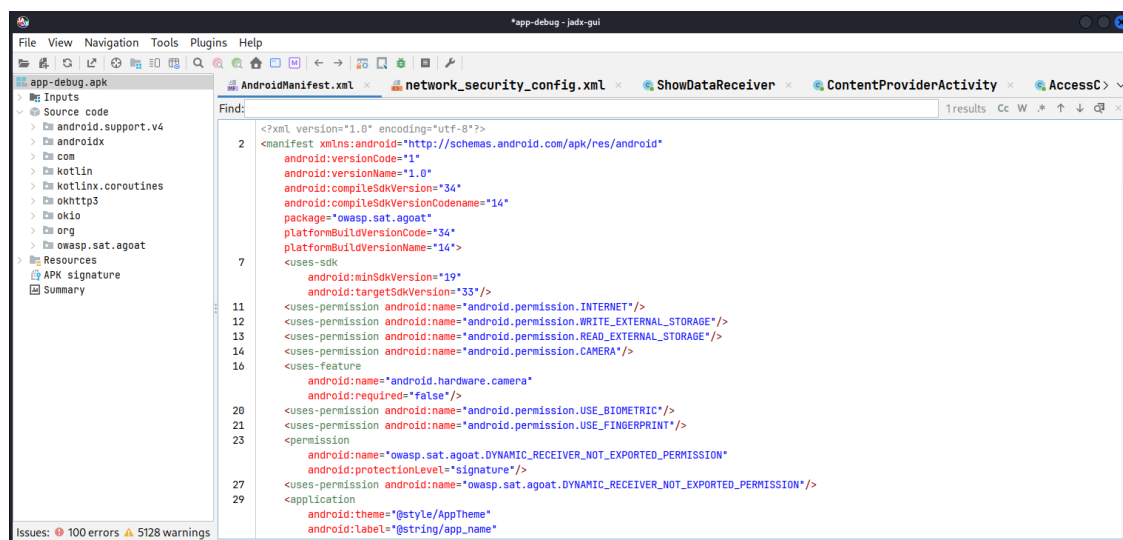


Figure 11. APK opened in JADX-GUI with the AndroidManifest.xml visible.

4.2 Manifest flags of interest

The manifest revealed several values that are intentionally insecure in this lab and should be avoided in production builds.

```
android:debuggable="true"
android:allowBackup="true"
android:networkSecurityConfig="@xml/network_security_config"
```

Observation	Why it matters	Production recommendation
debuggable=true	Makes runtime debugging easier.	Disable debugging in release builds.
allowBackup=true	May expose app data through backup workflows.	Disable backup for sensitive apps or control it carefully.
custom networkSecurityConfig	Defines trust anchors and cleartext traffic behavior.	Use a strict release configuration.

4.3 Network security configuration

The network security configuration allowed cleartext traffic and trusted user certificates. This is convenient for a lab because it makes Burp interception easier, but it would be risky in a production mobile app.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <network-security-config>
3   <base-config cleartextTrafficPermitted="true">
4     <trust-anchors>
5       <certificates src="system"/>
6       <certificates src="user"/>
7     </trust-anchors>
8   </base-config>
9   <domain-config>
10    <domain includeSubdomains="true">cve.org</domain>
11    <pin-set expiration="2050-01-01">
12      <pin digest="SHA-256">p4si93w2LTZdXU4LAa2VaonZLK406ZHmkjeAn5vXiHA=
13      </pin>
14      <pin digest="SHA-256">DxH4tt40L+eduF6szpY6T0N1xhZhBd+pJ9wbHLQ2fuw=
15      </pin>
16      <pin digest="SHA-256">++MBgDH5WGvL9Bcn5Be30cRcL0f50+NyoXuWtQdX1aI=
17      </pin>
18    </pin-set>
19  </domain-config>
20 </network-security-config>

```

Figure 12. network_security_config.xml viewed in JADX-GUI.

Mitigation

For release builds, set `cleartextTrafficPermitted=false` and normally trust only system certificate authorities, not user-installed CAs.

5. Exported Android components

Android components become external entry points when they are exported. Exporting a component is not automatically a vulnerability, but it becomes dangerous when the component exposes sensitive behavior without authentication or caller validation.

5.1 Exported Activity and deeplink

The activity `AccessControl1ViewActivity` was declared as exported and reachable through a deeplink using the scheme `androgoat` and host `vulnapp`.

```
adb shell am start -a android.intent.action.VIEW -d "androgoat://vulnapp"
```

```
<activity
    android:label="@string/activity"
    android:name="owasp.sat.agoat.AccessControllViewActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.VIEW"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <data
            android:scheme="androgoat"
            android:host="vulnapp"/>
    </intent-filter>
</activity>
```

Figure 13. Exported `AccessControl1ViewActivity` with a deeplink intent-filter.

Impact

If an exported activity displays sensitive content or assumes the user came from a trusted previous screen, a deeplink can become an access-control bypass.

5.2 Exported BroadcastReceiver

The manifest also showed an exported receiver named `ShowDataReceiver`. Exported receivers can be invoked from outside the app, so their behavior must be treated as part of the public attack surface.

```
adb shell am broadcast -n owasp.sat.agoat/.ShowDataReceiver
```

```
<receiver
    android:name="owasp.sat.agoat.ShowDataReceiver"
    android:enabled="true"
    android:exported="true"/>
```

Figure 14. ShowDataReceiver is exported in the manifest.

```
@Metadata(d1 = {"%u0000 %lnu0002%u0018%u0002%lnu0002%u0018%u0002%lnu0002%b%u0002%lnu0002%u0010%u000e%lnu0002%b%u0007%lnu0002%u0010%u0002%lnu0000%ln"}  
private final String aws_access_key_id = "AKIA7SAQKKOLPQ7HABC";  
  
builder.sendMessage("Connected to AWS account using Access key + this$0.aws_access_key_id + " and secret key + " + this$0.aws_secret_access_key);  
Log.d("Info", "Connected to AWS account using Access key + " + this$0.aws_access_key_id + " and secret key + " + this$0.aws_secret_access_key);
```

Figure 15. The receiver displays a username, password and key in cleartext.

5.3 Exported Service

The service `DownloadInvoiceService` was also tested as an externally triggerable component. A service can be risky if it performs sensitive background actions without checking who started it.

```
adb shell am startservice -n owasp.sat.agoat/.DownloadInvoiceService
```

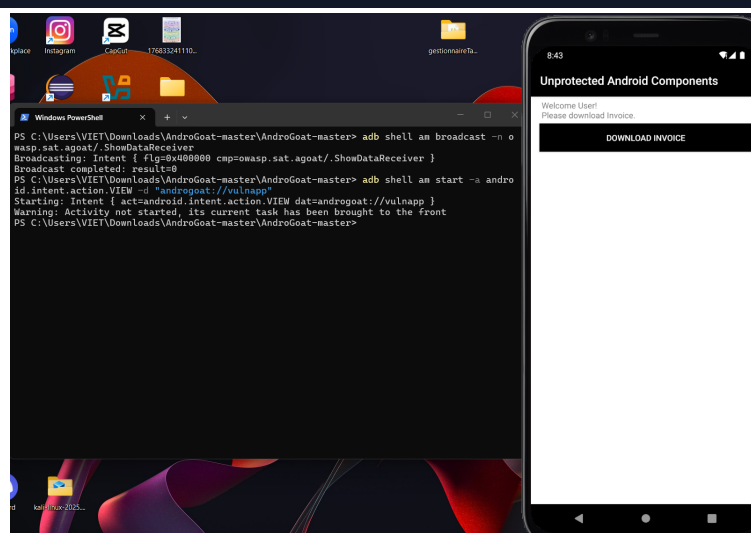


Figure 16. External service test using ADB and the corresponding app behavior.

Mitigation

Do not export activities, services or receivers unless needed. When external access is required, enforce permissions, validate the caller and repeat authorization checks inside the target component.

6. Hardcoded secrets and local data exposure

After reviewing the manifest, I searched the decompiled code for keywords such as secret, password, key, token and pin. This is a simple but effective beginner technique for quickly finding risky code paths.

6.1 Hardcoded cloud secrets

The class `CloudServicesActivity` contained cloud-style access key and secret key values directly in the application code. In a real app, this would be a high-impact issue because anyone can decompile an APK.

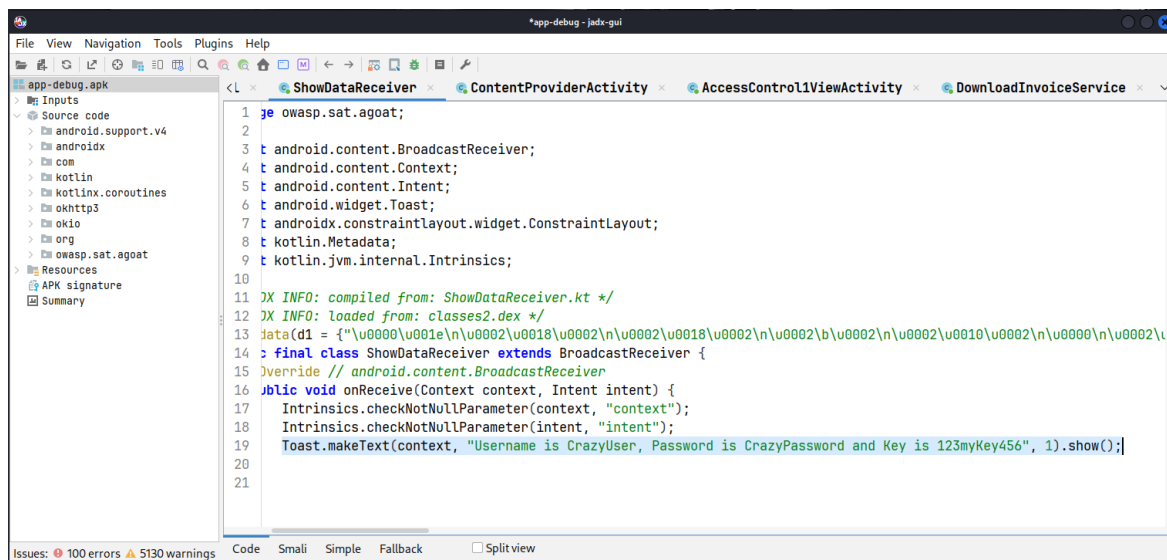


Figure 17. CloudServicesActivity showing cloud-style secrets in the decompiled code.

Mitigation

Never place cloud secrets inside a mobile client. Put privileged operations behind a backend, use short-lived scoped tokens, rotate exposed credentials and monitor cloud account activity.

6.2 Hardcoded username, password and key

The class `ShowDataReceiver` displayed a username, password and key in a Toast. This combines two problems: the receiver is exported and it leaks sensitive values.

```
Username is CrazyUser,  
Password is CrazyPassword,  
Key is 123myKey456
```

```
@Metadata(d1 = {"\u0000 \n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\b\u0002\n\u0002\u0010\u000e\n\u0002\b\u0007\n\u0002\u0010\u0002\n\u0000\u0000\n"}  
private final String aws_access_key_id = "AKIA56QKOLQP7G7ABC";  
builder.sendMessage("Connected to AWS account using Access key " + this$.aws_access_key_id + " and secret key " + this$.aws_secret_access_key);  
Log.d("[Info]", "Connected to AWS account using Access key " + this$.aws_access_key_id + " and secret key " + this$.aws_secret_access_key);
```

Figure 18. Sensitive values displayed by ShowDataReceiver.

6.3 Cleartext username/PIN pairs in a ContentProvider

I also reviewed a ContentProvider-related class that creates a user_pins table and inserts cleartext username/PIN pairs.

```
AndroGoat / AndroGoat
root      / toor
Admin     / Admin
```



Figure 19. ContentProviderActivity inserting cleartext username/PIN pairs.

Mitigation

Do not store passwords, keys or PINs in cleartext. Use Android Keystore, encrypted storage and server-side verification where appropriate. Protect ContentProviders with permissions and strict query validation.

7. Rooting the emulator with rootAVD

Root access is very useful for mobile security testing because it allows deeper inspection of app-private files, SQLite databases, shared preferences, caches and runtime behavior. It is also risky on real user devices, which is why this work was limited to a lab emulator.

7.1 Initial state: emulator not rooted

Before using rootAVD, I checked root status from two places: AndroGoat and ADB. AndroGoat reported that the device was not rooted, and the su command failed from the shell.

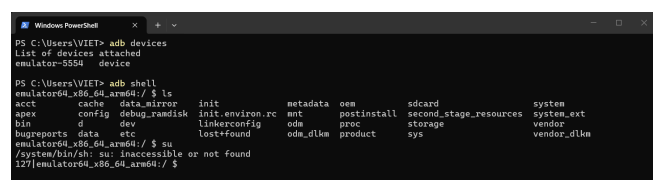
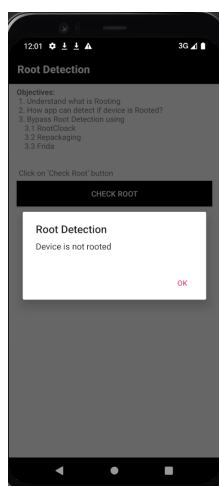


Figure 20. AndroGoat initially reports the device as not rooted.

Figure 21. The su command fails before rootAVD.

7.2 Activating root with rootAVD

After running rootAVD and restarting the emulator, I repeated the same checks. AndroGoat now detected root, and the ADB shell could obtain a root prompt.

```
adb shell
su
ls /
```

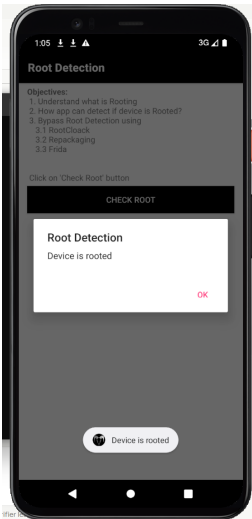


Figure 22. After rootAVD, AndroGoat detects the emulator as rooted.

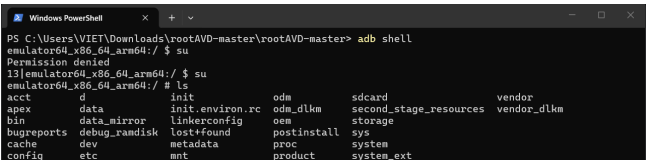


Figure 23. ADB root shell confirmed by the # prompt and system directory listing.

Key idea

The prompt change from \$ to # is an important practical signal: it means the shell now has root privileges.

8. Root detection theory

Root detection is a defensive signal. It tries to determine whether the app is running in a modified or higher-risk environment. But it does not prove that the app is secure, especially when the decision is made only inside the APK.

8.1 Common root indicators

Detection family	Typical check	Weakness
su binary	Look for /system/bin/su, /system/xbin/su or /sbin/su.	Files can be hidden, renamed, filtered or hooked.
Root packages	Check for Magisk, SuperSU, Xposed or LSPosed.	Package names can be hidden or filtered.
Build properties	Read test-keys, ro.debuggable or ro.secure.	Properties can be spoofed and are weak alone.
Writable paths	Try to write to protected paths or inspect mount flags.	Modern systemless root may avoid obvious modifications.

Detection family	Typical check	Weakness
Native checks	Use C/C++ access(), stat() or execve().	Still bypassable with native hooks or patching.
Remote attestation	Use server-side integrity verdicts.	Stronger, but must be designed carefully and is still one signal.

8.2 Typical beginner-level logic

```
boolean isRooted() {
    if (fileExists("/system/xbin/su")) return true;
    if (fileExists("/system/bin/su")) return true;
    if (canExecute("su")) return true;
    if (isPackageInstalled("com.topjohnwu.magisk")) return true;
    if (buildTagsContain("test-keys")) return true;
    return false;
}
```

Teaching point

This logic is understandable, but weak if it runs only on the client. A tester can influence what fileExists(), package checks or root-checking methods return.

9. Bypassing root detection with Frida

A root bypass does not necessarily remove root. It usually changes what the application sees or changes the application logic itself. The emulator can remain rooted while the app receives a controlled answer saying that it is not rooted.

9.1 Initial rooted state

Before injecting the Frida script, the app correctly detected the rooted emulator.

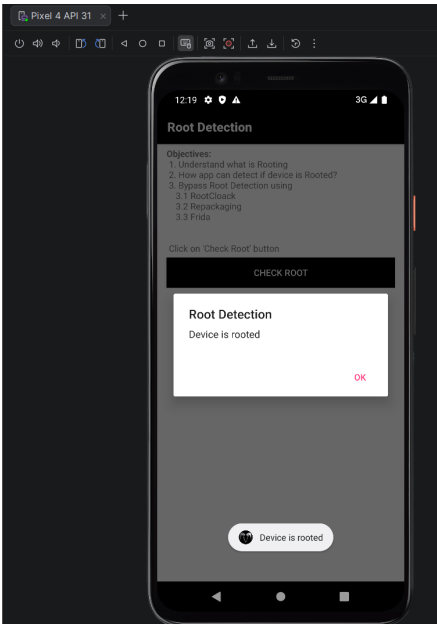


Figure 24. Initial Frida lab state: AndroGoat detects that the emulator is rooted.

9.2 Verifying Frida Server

Frida Server must run on the rooted emulator before the host can attach to the target process. I verified that the process was active using ADB.

```
adb shell ps -A | findstr frida
```

```
PS C:\Users\VIET\Desktop\AndroGoat_Repack> adb shell ps -A | findstr frida
root      12743    317 12925156 247740 0          S    frida-server
PS C:\Users\VIET\Desktop\AndroGoat_Repack> |
```

Figure 25. frida-server process visible on the emulator.

9.3 Injecting the bypass script

I attached Frida to the AndroGoat process by PID and loaded my JavaScript hook.

```
frida -U -p 12134 -l .\bypass_root.js
```

```
PS C:\Users\VIET\Desktop\AndroGoat_Repack> frida -U -p 12134 -l .\bypass_root.js
Frida 17.9.10 - A world-class dynamic instrumentation toolkit

Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit

More info at https://frida.re/docs/home/

Connected to Android Emulator 5554 (id=emulator-5554)

Attaching...
[*] Frida connecté !
[*] Bypass root detection en cours...
[*] Bypass actif ! Clique CHECK ROOT
[Android Emulator 5554::PID::12134 ]-> [+] isRooted() hookée -> FALSE
```

Figure 26. Frida attaches to the AndroGoat process and confirms that isRooted() is hooked.

9.4 How the hook works

The script waits for the Java runtime, targets `owasp.sat.agoat.RootDetectionActivity`, then replaces root detection methods so they return false. It also hooks `java.io.File.exists()` to hide paths containing root-related keywords.

```
Java.perform(function () {
  console.log("[*] Frida connected!");

  var RootDetection = Java.use("owasp.sat.agoat.RootDetectionActivity");

  RootDetection.isRooted.implementation = function () {
    console.log("[+] isRooted() hooked -> false");
    return false;
  };

  RootDetection.isRooted1.implementation = function () {
    console.log("[+] isRooted1() hooked -> false");
    return false;
  };

  var File = Java.use("java.io.File");
  File.exists.implementation = function () {
    var path = this.getPath();
    if (path.indexOf("su") >= 0 ||
        path.indexOf("magisk") >= 0 ||
        path.indexOf("superuser") >= 0) {
      console.log("[+] Hidden root path: " + path);
      return false;
    }
  };
});
```

```

    }
    return this.exists.call(this);
  };
});

```

```

PS C:\Users\VIET\Desktop\AndroGoat_Repack> cat .\bypass_root.js
Java.perform(function() {

    console.log("[*] Frida connectÃ@ !");
    console.log("[*] Bypass root detection en cours...");

    var RootDetection = Java.use("owasp.sat.agoat.RootDetectionActivity");

    RootDetection.isRooted.implementation = function() {
        console.log("[+] isRooted() hookÃ@e Ã+ FALSE");
        return false;
    };

    RootDetection.isRooted1.implementation = function() {
        console.log("[+] isRooted1() hookÃ@e Ã+ FALSE");
        return false;
    };

    var File = Java.use("java.io.File");
    File.exists.implementation = function() {
        var path = this.getPath();
        if (path.indexOf("su") >= 0 ||
            path.indexOf("magisk") >= 0 ||
            path.indexOf("superuser") >= 0) {
            console.log("[+] CachÃ@ : " + path);
            return false;
        }
        return this.exists.call(this);
    };

    console.log("[*] Bypass actif ! Clique CHECK ROOT");
});
PS C:\Users\VIET\Desktop\AndroGoat_Repack> |

```

Figure 27. Screenshot of the bypass_root.js script used during the lab.

9.5 Result: the app sees a non-rooted device

After injecting the script and pressing CHECK ROOT again, AndroGoat displayed Device is not rooted. This is the desired bypass result.

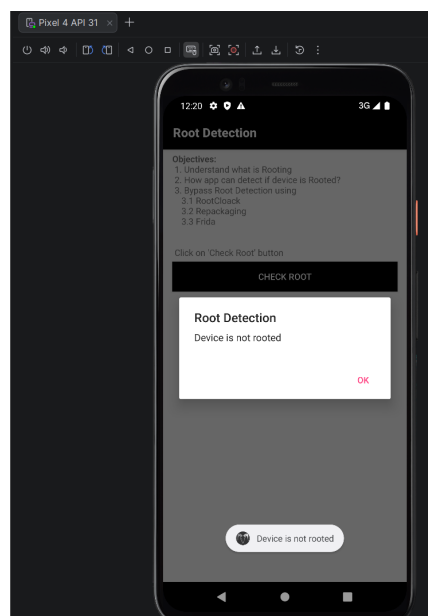


Figure 28. After Frida injection, AndroGoat considers the device not rooted.

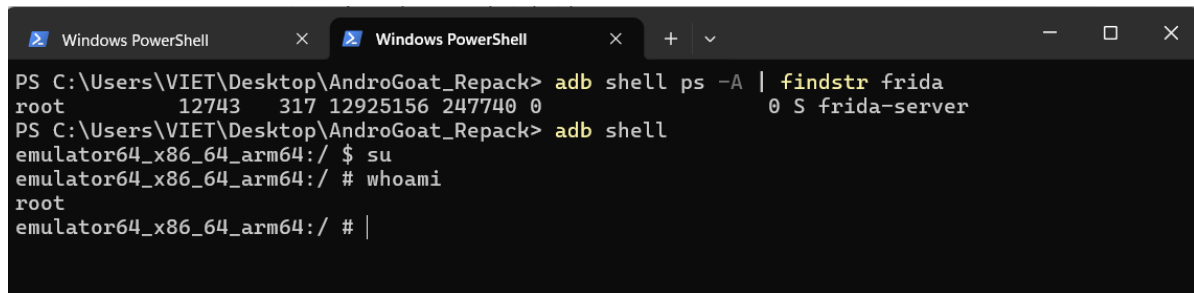
9.6 Confirming that the emulator is still rooted

The final verification is important: I checked from ADB that the emulator still had root access. This proves that Frida did not remove root; it only changed the answer seen by the app.

```

adb shell
su
whoami

```



```
PS C:\Users\VIET\Desktop\AndroGoat_Repack> adb shell ps -A | findstr frida
root      12743    317 12925156 247740 0          0 S frida-server
PS C:\Users\VIET\Desktop\AndroGoat_Repack> adb shell
emulator64_x86_64_arm64:/ $ su
emulator64_x86_64_arm64:/ # whoami
root
emulator64_x86_64_arm64:/ # |
```

Figure 29. ADB still returns root after the application-side bypass.

Core lesson

The real system state and the app's observed state can be different. With Frida, the emulator remains rooted, but AndroGoat receives a false result from its root-checking logic.

10. Findings summary

#	Finding	Evidence	Risk
1	HTTP traffic is interceptable	Request appears in Burp HTTP history	Requests, parameters and endpoints can be observed.
2	Cleartext traffic allowed	cleartextTrafficPermitted=true	Weak transport security if used in production.
3	User CAs trusted	certificates src=user	HTTPS interception easier on modified devices.
4	Debuggable build	android:debuggable=true	Runtime debugging and inspection easier.
5	Backup enabled	android:allowBackup=true	Possible local data extraction.
6	Exported deeplink activity	AccessControl1ViewActivity + androgoat://vulnapp	Potential access-control bypass.
7	Exported receiver leaks secrets	ShowDataReceiver displays username/password/key	Sensitive information disclosure.
8	Exported service	DownloadInvoiceService externally triggerable	Unauthorized action execution.
9	Hardcoded cloud secrets	CloudServicesActivity contains keys	Cloud credential exposure.
10	Cleartext PIN pairs	ContentProviderActivity inserts username/PIN values	Local sensitive data exposure.
11	Client-side root check bypassable	Frida forces isRooted() to false	Local-only protection can be bypassed.

11. Remediation recommendations

11.1 Secure the manifest

```
android:debuggable="false"
android:allowBackup="false"
```

```
android:exported="false"    // when external access is not required
```

Exported components that remain necessary should enforce permissions, validate the caller and repeat authorization checks inside the component itself.

11.2 Harden network security

Sensitive apps should reject cleartext traffic in release builds and avoid trusting user-installed certificate authorities unless the use case requires it.

11.3 Remove client-side secrets

Secrets inside an APK are not secret. Mobile clients should not contain cloud access keys, privileged API secrets or long-lived credentials. Sensitive operations should move to a backend with short-lived, scoped tokens.

11.4 Protect local data

Passwords, tokens, PINs and personal data should not be stored in cleartext. Depending on the threat model, use Android Keystore, encrypted preferences, encrypted databases, hashing for verification and strict ContentProvider permissions.

11.5 Treat root detection as one signal

Root detection can improve risk awareness, but it should not be the only security boundary. Stronger designs combine local checks, app integrity, device integrity and server-side decisions.

Practical rule Do not trust only the client. If a security decision is made only inside the APK, assume it can be inspected, patched or hooked.

12. Lessons learned

This project helped me connect multiple mobile security skills into one workflow. The most important lesson is that Android pentesting is not a single tool. It is a sequence of checks that complement each other.

Technique	What it helped me understand
ADB	How to install apps, start components, configure the device and verify root state.
Burp Suite	How to observe app traffic and validate proxy/certificate setup.
JADX-GUI	How to inspect the manifest, decompiled code and resources.
Manifest review	How exported components and dangerous flags expand the attack surface.
Keyword search	How secrets often appear in client code when developers take shortcuts.
rootAVD	How root enables deeper app inspection and prepares advanced testing.
Frida	How runtime instrumentation can change what the app observes without modifying the APK.

The strongest concept from the Frida part is the difference between the real device state and the state observed by the application. The emulator was still rooted, but the app believed it was not rooted because the root-checking methods were hooked at runtime.

Conclusion

This personal AndroGoat assessment covered a complete beginner-to-intermediate Android mobile security workflow. I started with lab setup and APK installation, then moved into traffic interception, static analysis, exported component testing, hardcoded secret discovery, emulator rooting and Frida-based root detection bypass.

The exercise showed why mobile app security needs both static and dynamic analysis. Burp Suite reveals communication behavior. JADX reveals what is inside the APK. ADB gives device-level control. rootAVD enables deeper inspection. Frida demonstrates how runtime logic can be influenced.

The final takeaway is simple: a secure mobile app must protect its network layer, local storage, exported components, secrets and runtime decisions. Good mobile security is layered, server-aware and designed with the assumption that the client can be inspected.

Portfolio-ready project summary

I performed a full Android mobile security assessment of AndroGoat, an intentionally vulnerable Kotlin application. The project covered lab setup, APK installation, Burp Suite traffic interception, JADX-GUI static analysis, AndroidManifest review, exported component testing, hardcoded secret discovery, emulator rooting with rootAVD and runtime root-detection bypass using Frida. I identified insecure network configuration, exported components, hardcoded credentials, cleartext local data and bypassable client-side root checks, then documented practical remediation strategies for each issue.

References and further reading

- OWASP Mobile Application Security Testing Guide (MASTG)
- Android Developers - Application fundamentals and App manifest overview
- Frida - Dynamic instrumentation toolkit
- rootAVD - Root Android Studio AVDs with Magisk
- Apktool - Reverse engineering Android APK files
- Google Play Integrity API overview
- AndroGoat - OWASP intentionally vulnerable Android application